

Ichrak BCHIR
Zina BENSADOK
Kafine DIAKITE
Celia HAKMI
Ines ADDAM

Rapport des Fantastiques

Projet Robotique UL2IN013

I - Contexte

L'intelligence artificielle et la robotique progressent rapidement, rendant les systèmes autonomes de plus en plus accessibles. Derrière ces systèmes, il y a avant tout des logiciels s'appuyant sur des architectures modulaires, des algorithmes et des outils de simulation permettant de tester des comportements sans risque.

Ce projet s'inscrit dans cette démarche. Nous avons conçu et développé en Python un environnement de simulation complet pour un robot mobile à deux-roues, comprenant une plateforme avec gestion des obstacles et détection de collisions, une interface graphique via Pygame, ainsi qu'un ensemble de stratégies de déplacement pilotées par un contrôleur. L'architecture logicielle a été pensée pour être à la fois modulaire et extensible, facilitant ainsi l'intégration de nouvelles fonctionnalités et le passage de la simulation au robot réel via un système d'adaptateurs.

II - Mode d'emploi

1. Prérequis

Python 3 installé sur votre système (Linux, macOS ou Windows).

Les bibliothèques nécessaires : Pygame (pour la visualisation) .

2. Installation

1. Télécharger le projet depuis le dépôt GitHub :
git clone <https://github.com/celiahakmi/les-fantastiques.git>
2. Se placer dans le dossier du projet :
cd les-fantastiques
3. Installez les bibliothèques :
pip install pygame

3. Lancement du programme

- Mode simulation (par défaut)

Par défaut, le programme s'exécute en mode simulation. Pour le lancer, utiliser la commande suivante : **python main.py**

Une fenêtre graphique s'ouvre alors, permettant de visualiser le comportement du robot via l'interface Pygame.

- Mode robot réel

Pour exécuter le programme sur le robot physique :

1. Ouvrir le fichier main.py dans un éditeur de texte
2. Modifier la ligne suivante :

simu = True en :

simu = False

3. Enregistrer le fichier
4. Relancer le programme avec la commande : **python main.py**

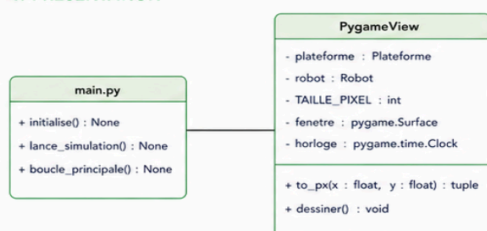
III - Schéma UML

Ce document présente le diagramme de classes UML d'un système de contrôle de robot . Il met en évidence l'organisation globale du projet en quatre parties principales : la présentation graphique, le contrôleur (stratégies), le modèle de simulation, et les adaptateurs.

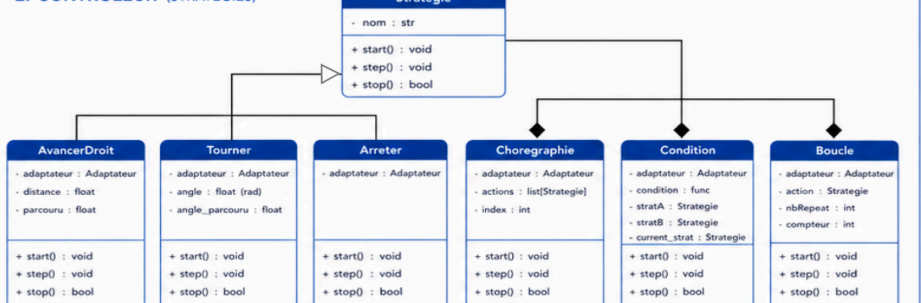
Les stratégies permettent de définir différents comportements du robot de manière modulaire, tandis que les adaptateurs assurent l'interface entre ces stratégies et le robot, qu'il soit simulé ou réel

DIAGRAMME DE CLASSE UML

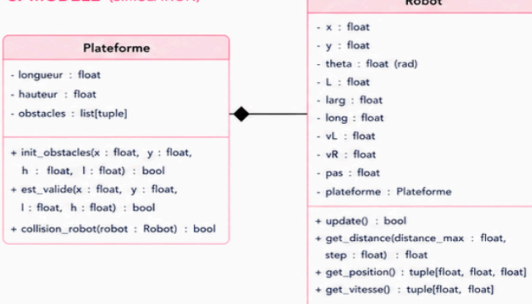
1. PRÉSENTATION



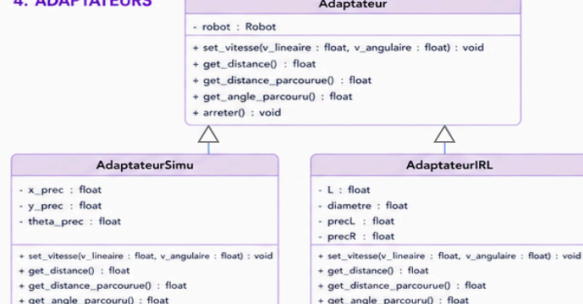
2. CONTRÔLEUR (STRATÉGIES)



3. MODÈLE (SIMULATION)



4. ADAPTATEURS

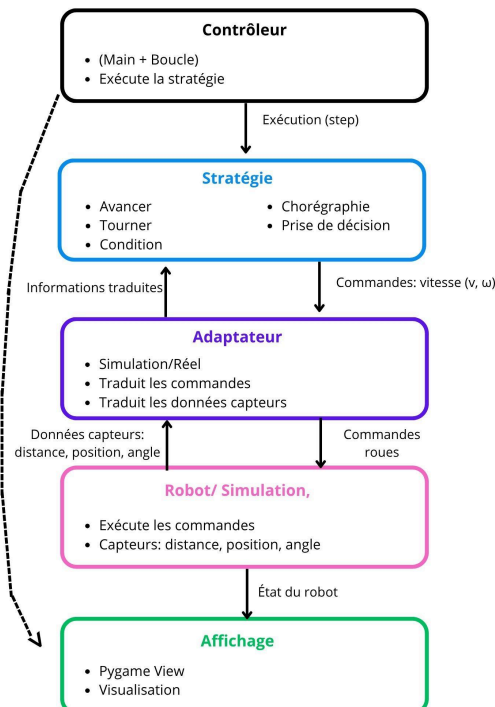


LÉGENDE



IV - Schéma Fonctionnel

Ce schéma fonctionnel présente l'organisation globale du système de contrôle du robot. Il met en évidence les principaux blocs ainsi que les échanges d'informations entre eux. Le contrôleur dirige l'exécution des stratégies, qui prennent les décisions de déplacements du robot. Ces stratégies envoient des commandes grâce à un adaptateur qui traduit ces instructions vers le robot (réel ou simulé). Ensuite, le robot va exécuter ces actions et renvoyer des informations via les capteurs (distance, position). Ces informations vont, ensuite, être utilisées par les stratégies pour qu'elles puissent adapter leur comportement en temps réel.



V- Équations régissant le mouvement

Le robot simulé dans ce projet dans la classe Robot de notre fichier simulation, est un robot mobile à deux roues indépendantes sans roue directrice. Le déplacement du robot dans le plan est décrit par trois variables d'état principales : sa position (x,y) dans l'espace, ainsi que son orientation θ qui représente l'angle du robot par rapport à l'axe horizontal.

Le mouvement est entièrement déterminé par les vitesses des deux roues, notées v_L pour la roue gauche et v_R pour la roue droite. À partir de ces variables, on a défini deux types de vitesses permettant de décrire le déplacement du robot.

Tout d'abord, la vitesse linéaire du robot correspond à la vitesse "vers l'avant" du robot, c'est-à-dire à quelle vitesse son centre se déplace dans la direction où il est orienté. Elle est donnée par la moyenne des vitesses des deux roues : $v = (v_L + v_R) / 2$

Ensuite, nous avons implémenté la vitesse angulaire qui décrit le mouvement de rotation du robot sur lui-même. Elle dépend de la différence de vitesse entre les deux roues et de l'écartement L entre celles-ci : $\omega = (v_R - v_L) / L$

Lorsque les deux roues tournent à la même vitesse, le robot avance en ligne droite. À l'inverse, une différence de vitesse entre les roues provoque une rotation.

À partir de ces deux grandeurs, l'évolution de la position du robot est calculé par les équations suivantes, appliquées de manière discrète à chaque pas de simulation :

```
self.x += v * math.cos(self.theta) * self.pas
```

```
self.y += v * math.sin(self.theta) * self.pas
```

```
self.theta += \omega * self.pas
```

Ces équations implémentées dans la fonction de mise à jour du robot `update()`, permettent à celui-ci, d'avancer dans la direction de son orientation actuelle tout en pouvant simultanément tourner.

VI - Proposition d'amélioration

Bien que notre architecture actuelle soit fonctionnelle, deux axes d'amélioration permettraient d'étendre les capacités de notre simulateur :

- Simulation d'un flux vidéo par rendu 3D (Ray-Casting). L'objectif est de passer d'une connaissance spatiale à une perception sensorielle totale. Actuellement, notre simulateur traite des données de distance via la méthode `get_distance()` de l'adaptateur. Il serait intéressant d'ajouter un affichage 3D à notre interface. En projetant ces distances en perspective, nous pourrions générer une matrice de pixels simulant en temps réel le flux vidéo de la caméra du robot réel. Cela permettrait au robot de "voir" son environnement virtuel au lieu de simplement le mesurer.

- Couplage avec des algorithmes de vision par ordinateur. Une fois ce flux vidéo simulé, l'étape suivante est son traitement algorithmique pour la prise de décision. Nous pourrions implémenter des stratégies de planification à long terme basées sur la reconnaissance de formes ou de couleurs (pour des tâches de suivi de balise ou de "chasse au trésor"). Le robot ne se contenterait plus d'éviter des obstacles, mais analyserait la position relative d'une cible sur l'image pour ajuster les vitesses des moteurs via l'Adaptateur.

VI - Conclusion

Ce projet nous a permis de développer un logiciel de robotique, alliant une simulation physique à une architecture logicielle modulaire et évolutive. En séparant la prise de décision, la gestion des commandes via les adaptateurs et la visualisation graphique, nous avons réussi à créer un système capable de fonctionner en environnement virtuel mais également sur un robot réel. Cette expérience nous a non seulement permis de maîtriser les principes de la programmation orientée objet en Python, mais aussi d'appréhender les défis de l'autonomie robotique et de la gestion de projet.